

JSON Cheat Sheet

Concetto, storia, regole pratiche di sintassi, tipi di dato ed esempi (oggetti, liste, strutture annidate), la famiglia JSON, JSON Schema e il ruolo di JSON con l'intelligenza artificiale.

Indice

- 1 · Che cos'è JSON
- 2 · Storia ed evoluzione
- 3 · Regole di sintassi
- 4 · I tipi di dato
- 5 · Strutture ed esempi progressivi
- 6 · Pattern e idiomi comuni
- 7 · Errori frequenti (do / don't)
- 8 · La famiglia JSON
- 9 · JSON Schema (la validazione)
- 10 · JSON e l'intelligenza artificiale
- 11 · Lavorare con JSON in Python
- 12 · JSON come base di dati
- 13 · Sviluppi futuri
- 14 · Riferimento rapido
- 15 · Fonti e riferimenti

1 Che cos'è JSON

JSON (JavaScript Object Notation) è un formato di interscambio dati **testuale**, leggero e indipendente dal linguaggio. Serve a rappresentare dati strutturati come testo, per scambiarli tra sistemi o salvarli su file.

Pur derivando dalla sintassi degli oggetti letterali di JavaScript, JSON è uno standard a sé: praticamente ogni linguaggio (Python, C#, Java, Rust, JavaScript...) lo legge e lo scrive. È pensato per essere al tempo stesso **leggibile dall'uomo** e **facile da elaborare per le macchine**.

Carta d'identità del formato

Proprietà	Valore
Tipo	Formato dati testuale (non un linguaggio di programmazione)
Estensione file	.json
MIME type	application/json
Codifica	UTF-8 (raccomandata e di fatto obbligatoria per l'interscambio)
Standard	RFC 8259 / STD 90 ed ECMA-404
Usi tipici	API web (REST), file di configurazione, log, messaggi, persistenza dati

IN SINTESI

JSON descrive dati come combinazione di due sole strutture — **oggetti** (coppie chiave/valore) e **array** (liste ordinate) — annidabili a piacere. Tutto qui: la sua forza è la semplicità.

2 Storia ed evoluzione

JSON viene formalizzato da **Douglas Crockford** all'inizio degli anni 2000 (sito json.org, 2002) come alternativa più leggera a XML per lo scambio dati nelle prime applicazioni web. Nasce dalla notazione degli oggetti di JavaScript, ma viene presto adottato ovunque proprio perché indipendente dal linguaggio.

La grammatica è volutamente **minimale e stabile**: una scelta di design esplicita ("JSON non cambierà"). Le revisioni successive degli standard hanno chiarito dettagli (es. codifica, valore di primo livello) senza stravolgere il formato. Negli anni JSON ha di fatto **soppiantato XML** come formato dominante per le API.

Tappe di standardizzazione

Anno	Tappa	Significato
~2001-2002	json.org	Crockford definisce e pubblica il formato
2006	RFC 4627	Prima specifica IETF e registrazione del MIME type
2013	ECMA-404	Standardizzazione della sola grammatica (sintassi)
2014	RFC 7159	Aggiornamento: qualsiasi valore può stare al primo livello
2017	RFC 8259 / STD 90	Versione vigente; impone UTF-8 per l'interscambio

Nota: ECMA-404 descrive *solo* la sintassi; RFC 8259 aggiunge i requisiti d'interoperabilità (codifica, raccomandazioni). Le due specifiche sono allineate e coese.

3 Regole di sintassi

JSON si costruisce con due sole strutture contenitore e sette tipi di valore. Le regole sono poche ma **rigide**: ciò che non è esplicitamente permesso non è valido.

Le due strutture

- **Oggetto** `{ }`: insieme *non ordinato* di coppie `"chiave": valore`, separate da virgola.
- **Array** `[ ]`: lista *ordinata* di valori, separati da virgola.

Regole pratiche fondamentali

Cosa è permesso e cosa no

Regola	Permesso	Vietato
Le chiavi sono stringhe tra doppi apici	"chiave"	chiave · 'chiave'
Le stringhe usano solo i doppi apici	"testo"	'testo'
Virgola tra gli elementi, mai dopo l'ultimo	[1, 2]	[1, 2,]
Commenti	—	// ... /* ... */
Numeri: niente zero iniziale, niente <code>+</code>	0.5 · -7 · 1e3	.5 · 030 · +7
Valori non numerici speciali	—	NaN · Infinity
Spazi/indentazione tra i token	irrilevanti	—

PUNTI SPESSO TRASCURATI

Ordine delle chiavi: in un oggetto non è significativo (gli array invece sono ordinati). **Chiavi duplicate**: la specifica le sconsiglia e il comportamento è dipendente dall'implementazione (di norma "vince l'ultima"); meglio evitarle sempre. **Codifica**: UTF-8, senza BOM per l'interscambio.

4 I tipi di dato

Un valore JSON è uno di sette tipi: i quattro *primitivi* (stringa, numero, booleano, null) e i due *strutturati* (oggetto, array).

I tipi di valore

Tipo	Descrizione	Esempio
string	Testo Unicode tra doppi apici	"Cesena"
number	Intero o decimale, anche notazione esponenziale	42 · -3.14 · 6.022e23
boolean	Valore logico	true · false
null	Assenza di valore	null
object	Coppie chiave/valore	{ "k": 1 }
array	Lista ordinata di valori	[1, 2, 3]

Sequenze di escape nelle stringhe

Escape	Significato	Escape	Significato
\"	doppio apice	\n	a capo (newline)
\\	backslash	\t	tabulazione
\/	slash (opzionale)	\r	ritorno carrello
\b	backspace	\f	form feed
\uXXXX	carattere Unicode dato il codice esadecimale a 4 cifre (es. \u00e8 = è)		

ATTENZIONE AI NUMERI

JSON non distingue interi e decimali a livello di formato, ma in pratica i parser usano il **doppio precisione IEEE 754**. Conseguenze: `0.1 + 0.2` non è esatto, e **gli interi molto grandi** (ID, timestamp in ns) possono perdere precisione oltre i $\sim 2^{53}$. Regola pratica: ID e importi monetari vanno trasmessi come **stringhe**.

5 Strutture ed esempi progressivi

Dal caso minimo alle strutture annidate. Codice colorato: **chiavi**, **stringhe**, **numeri**, **true/false/null**.

Valore di primo livello

Dal 2014 (RFC 7159) **qualsiasi** valore è un documento JSON valido, non solo oggetti o array.

Valore al primo livello	Documento JSON valido
stringa	"Ciao"
numero	42
booleano	true
null	null

Oggetto semplice

```
{
  "nome": "Mario",
  "eta": 30,
  "attivo": true
}
```

Oggetto con tutti i tipi

```
{
  "stringa": "testo",
  "intero": 42,
  "decimale": 3.14,
  "negativo": -7,
  "esponenziale": 6.022e23,
  "booleano": false,
  "nullo": null,
  "lista": [1, 2, 3],
  "oggetto": { "chiave": "valore" }
}
```

Array di primitivi (lista)

```
["rosso", "verde", "blu"]
```

Array di oggetti

```
[
  { "id": 1, "nome": "Anna" },
  { "id": 2, "nome": "Luca" }
]
```

Oggetti e array annidati

```
{
  "azienda": "Acme Corporation",
  "sede": {
    "citta": "Cesena",
    "provincia": "FC",
    "cap": "47521"
  },
  "reparti": [
    { "nome": "Automazione", "robot": ["YRC1000", "GP25"] },
    { "nome": "Software", "robot": [] }
  ]
}
```

Esempio realistico: file di configurazione

```
{
  "plc": {
    "modello": "YRC1000",
    "ip": "192.168.1.10",
    "porta": 502,
    "timeout_ms": 1500,
    "abilitato": true
  },
  "inverter": {
    "modello": "GA500",
    "frequenze_hz": [30, 100],
    "rampa_s": 2.5
  },
  "segnali_io": [
    { "nome": "START", "indirizzo": "X0", "tipo": "input" },
    { "nome": "MOTORE", "indirizzo": "Y0", "tipo": "output" }
  ],
  "note": null
}
```

Esempio realistico: risposta di una API

```
{
  "status": "ok",
  "page": 1,
  "total": 2,
  "data": [
    {
      "id": "a1b2",
      "timestamp": "2026-06-02T10:15:30Z",
      "valori": { "temp": 21.4, "umidita": 47 },
      "tags": ["sensore", "linea-1"]
    }
  ]
}
```


6 Pattern e idiomi comuni

JSON non ha tipi per date, mappe o dati binari: si usano **convenzioni** consolidate.

Come rappresentare i casi ricorrenti

Esigenza	Convenzione	Esempio
Data / ora	Stringa in formato ISO 8601 (UTC con "Z")	"2026-06-02T10:15:30Z"
Mappa chiave→valore	Oggetto con chiavi dinamiche	{ "X0": true, "Y0": false }
Insieme / collezione	Array	["a", "b", "c"]
Enumerazione	Stringa da un insieme noto	"output"
ID / numeri grandi	Stringa (evita perdita di precisione)	"900719925474099"
Importo monetario	Stringa o interi in centesimi	"19.90" / 1990
Dato binario	Stringa Base64	"iVBORw0KGgo..."
Valore assente	null, campo mancante, o array vuoto	null / []

NULL VS MANCANTE VS VUOTO

Sono tre cose diverse: `"note": null` (presente ma senza valore), assenza della chiave `note` (non fornita), e `"note": ""` oppure `[]` (presente e vuota). Decidi una semantica e mantienila coerente in tutto il progetto.

7 Errori frequenti (do / don't)

Gli errori più comuni derivano dall'abitudine alla sintassi di JavaScript o Python, più permissiva di JSON.

Non valido

```
{
  'nome': 'Mario',
  "eta": 030,
  "saldo": .5,
  "lista": [1, 2, 3,],
}
```

In rosso i token che un parser JSON rifiuta: apici singoli, commenti, zero iniziale (030), decimale senza zero (.5) e virgola finale.

Corretto

```
{
  "nome": "Mario",
  "eta": 30,
  "saldo": 0.5,
  "lista": [1, 2, 3]
}
```

Checklist degli errori tipici

Errore	Correzione
Apici singoli per chiavi o stringhe	Usare sempre i doppi apici
Virgola dopo l'ultimo elemento	Rimuoverla
Commenti <code>//</code> o <code>/* */</code>	Non ammessi: spostarli fuori dal JSON (o usare JSON5/JSONC)
Chiavi senza apici	Racchiuderle tra doppi apici
<code>NaN</code> , <code>Infinity</code> , <code>undefined</code>	Usare null o una stringa convenzionale
Numeri come <code>.5</code> , <code>5.</code> , <code>030</code>	Scrivere 0.5, 5.0, 30
Caratteri di controllo non-escaped nelle stringhe	Usare <code>\n</code> , <code>\t</code> , ecc.

8 La famiglia JSON

Attorno a JSON è cresciuto un ecosistema di varianti e formati correlati, ciascuno per un'esigenza specifica.

Varianti e formati correlati

Formato	A cosa serve	Caratteristica chiave
JSON5	File di configurazione human-friendly	Commenti, virgole finali, apici singoli, chiavi senza apici
JSONC	Config con commenti (es. VS Code)	JSON + commenti <code>//</code> e <code>/* */</code>
NDJSON / JSON Lines	Streaming, log, big data	Un oggetto JSON completo per riga
JSON Pointer (RFC 6901)	Indirizzare un nodo interno	<code>/plc/ip</code>
JSON Patch (RFC 6902)	Descrivere modifiche a un documento	Operazioni add / remove / replace
Merge Patch (RFC 7386)	Aggiornamenti parziali semplici	Invio del solo "delta"
JSON Schema	Validare struttura e tipi	Vedi sezione 9
GeoJSON / JSON-LD	Dati geospaziali / dati collegati	Profili JSON per domini specifici
BSON · MessagePack · CBOR	Serializzazione binaria	Più compatti/veloci, non leggibili a occhio

JSON5 (esempio)

```
{
  // commento ammesso in JSON5
  nome: "Mario",
  eta: 30,
  lista: [1, 2, 3,],
  apici: 'singoli',
}
```

NDJSON / JSON Lines (esempio)

```
{"id": 1, "evt": "start"}
{"id": 2, "evt": "stop"}
{"id": 3, "evt": "reset"}
```

In NDJSON ogni riga è un documento JSON indipendente: ideale per leggere/scrivere flussi grandi senza caricare tutto in memoria.

9 JSON Schema (la validazione)

JSON Schema è esso stesso un documento JSON che descrive la *forma attesa* di altri JSON: tipi, campi obbligatori, vincoli. È il "contratto" tra chi produce e chi consuma i dati, e — come vedremo — tra software e IA.

La versione di riferimento attuale è il **Draft 2020-12** (il **Draft 7** è ancora molto diffuso); è in arrivo una versione **stabile "v1/2026"** che abbandona l'etichetta "draft" e introduce garanzie di compatibilità. Il dialetto si dichiara con la chiave `$schema`.

Schema che valida l'oggetto della sezione 5

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "nome": { "type": "string" },
    "eta": { "type": "integer", "minimum": 0 },
    "attivo": { "type": "boolean" }
  },
  "required": ["nome", "eta"],
  "additionalProperties": false
}
```

Parole chiave essenziali

Keyword	Significato
type	Tipo atteso: object, array, string, number, integer, boolean, null
properties	Schema di ciascun campo di un oggetto
required	Elenco dei campi obbligatori
items	Schema degli elementi di un array
enum	Insieme chiuso di valori ammessi
minimum / maximum	Limiti numerici
minLength / pattern	Vincoli su stringhe (lunghezza, regex)
additionalProperties	Se ammettere campi non dichiarati (spesso false)

10 JSON e l'intelligenza artificiale

JSON è diventato la **lingua franca** tra i modelli di IA e il software: è il formato con cui un LLM dichiara una chiamata a strumenti (*function calling / tool use*), restituisce dati strutturati e dialoga con gli agenti e con protocolli come MCP. Il problema storico — "il modello a volte risponde con testo invece che con JSON valido" — è oggi risolto a livello di generazione.

Tre livelli di affidabilità

Livello	Come funziona	Garanzia
1 · Prompt	"Rispondi in JSON con questi campi..."	Debole: funziona spesso, fallisce nei casi limite
2 · Function calling	Si definisce uno schema, il modello lo "chiama"	Buona: lo schema è un suggerimento, non un vincolo
3 · Structured output	Decodifica vincolata da JSON Schema	Massima: output schema-valido garantito

Il livello 3 si basa sul **constrained decoding** (decodifica vincolata): a ogni token generato il sistema *maschera* i token che violerebbero la grammatica/lo schema, tipicamente con un automa a stati finiti. Così la sintassi è valida per costruzione, non controllata "dopo". OpenAI ha introdotto gli Structured Outputs nel 2024, Anthropic la decodifica vincolata per Claude a fine 2025, mentre Gemini ha unificato JSON mode e structured output. Motori open source come **XGrammar** e **llguidance** rendono il sovraccarico quasi nullo.

Schema di uno strumento (definizione lato IA)

```
{
  "name": "leggi_temperatura",
  "description": "Legge la temperatura di un sensore",
  "input_schema": {
    "type": "object",
    "properties": {
      "sensore_id": { "type": "string" },
      "unita": { "type": "string", "enum": ["C", "F"] }
    },
    "required": ["sensore_id"]
  }
}
```

Output strutturato prodotto dal modello (schema-valido)

```
{
  "sensore_id": "S-12",
  "unita": "C",
  "valore": 21.7
}
```

DETTAGLIO PRATICO

Non tutti i vincoli sono imposti dal decoder: alcuni SDK **rimuovono** dallo schema vincoli come `minimum`, `maximum`, `pattern` (li spostano nella descrizione) e li verificano *dopo* la generazione, con eventuale retry. Lo schema garantisce struttura e tipi; sui valori puntuali, validare comunque a valle. Strumenti come Pydantic o Zod aiutano a definire lo schema e a tipizzare il risultato.

11 Lavorare con JSON in Python

Quasi ogni linguaggio sa leggere e scrivere JSON. In Python si usa il modulo standard `json` (incluso, nessuna installazione). Quattro funzioni coprono il 90% dei casi.

Le quattro funzioni fondamentali del modulo `json`

Funzione	Direzione	Uso
<code>json.load(f)</code>	file JSON → oggetto Python	Leggere un file
<code>json.dump(o, f)</code>	oggetto Python → file JSON	Scrivere un file
<code>json.loads(s)</code>	stringa JSON → oggetto Python	Da testo (es. risposta HTTP)
<code>json.dumps(o)</code>	oggetto Python → stringa JSON	Verso testo

Regola mnemonica: la "s" finale sta per *string*. `load/dump` lavorano su file; `loads/dumps` su stringhe.

```
import json

# Lettura da file -> dict / list Python
with open("config.json", encoding="utf-8") as f:
    dati = json.load(f)

# Scrittura su file (forma leggibile)
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(dati, f, indent=2, ensure_ascii=False)

# Conversione con stringhe (non file)
testo = json.dumps(dati, indent=2, ensure_ascii=False)
dati = json.loads(testo)
```

Corrispondenza dei tipi JSON ↔ Python

JSON	Python
object	dict
array	list
string	str
number (intero)	int
number (decimale)	float
true / false	True / False
null	None

Opzioni utili di `dump` / `dumps`

Opzione	Effetto
<code>indent=2</code>	Indentazione leggibile (per file di lettura/debug)
<code>ensure_ascii=False</code>	Mantiene gli accenti (è, à) invece di <code>\uXXXX</code>
<code>sort_keys=True</code>	Ordina le chiavi alfabeticamente
<code>separators=(",", ":")</code>	Minifica: nessuno spazio (per trasmissione/produzione)
<code>default=fn</code>	Funzione per serializzare tipi non nativi

Serializzare tipi non nativi (date, Decimal)

JSON non conosce `datetime` né `Decimal`: passandoli direttamente, `json.dumps` solleva `TypeError`. Si risolve con il parametro `default`, che indica come convertirli.

```
import json
from datetime import datetime, date
from decimal import Decimal

def encoder(o):
    if isinstance(o, (datetime, date)):
        return o.isoformat()          # "2026-06-02T10:15:30"
    if isinstance(o, Decimal):
        return str(o)                 # "19.90"
    raise TypeError(f"Tipo non serializzabile: {type(o)}")

record = {"ts": datetime.now(), "prezzo": Decimal("19.90")}
testo = json.dumps(record, default=encoder, ensure_ascii=False)
```

SICUREZZA

Usare sempre `json` (non `eval`) per interpretare testo JSON: `json.loads` analizza solo dati, mentre `eval` eseguirebbe codice arbitrario. Validare comunque l'input ricevuto da fonti esterne.

12 JSON come base di dati per un'applicazione

Per piccole applicazioni, prototipi, file di configurazione o cache, un singolo file JSON può fungere da "database": si carica in memoria, si modifica, si riscrive. È un approccio usato anche in produzione per dati di tipo configurazione (per esempio sistemi che gestiscono impostazioni di decine di punti vendita salvandole in file JSON), purché se ne rispettino i limiti.^[8]

Quando ha senso (e quando no)

JSON come archivio dati: criterio di scelta

Un file JSON basta se...	Meglio un vero DB (es. SQLite) se...
i dati sono piccoli e stanno in memoria	il dataset è grande: leggere un solo valore impone di caricare l'intero file ^[6]
le scritture sono poche, a cadenza umana	servono scritture frequenti o ad alto volume
un solo processo scrive per volta	più processi/thread scrivono insieme (i file JSON non gestiscono la concorrenza in scrittura) ^[5]
prototipo, configurazione, cache	servono tipi forti, vincoli, query e indici (JSON non impone tipi né struttura) ^[6]

IN BREVE

SQLite è incluso in Python, sta in un solo file ed è transazionale (ACID): è la naturale "promozione" quando un file JSON non basta più.^[6] Finché i dati sono piccoli e le scritture rare, JSON resta la via più semplice.

Il file dati di partenza

Un possibile schema: un oggetto radice con metadati e una lista di record.

```
{
  "_meta": { "versione": 1 },
  "dispositivi": [
    { "id": 1, "nome": "PLC-Linea1", "ip": "192.168.1.10", "attivo": true },
    { "id": 2, "nome": "Inverter-Pompa", "ip": "192.168.1.20", "attivo": false }
  ]
}
```

Un modulo CRUD minimale (create, read, update, delete)

Le due funzioni di base — `carica()` e `salva()` — leggono e riscrivono l'intero file; sopra di esse si costruiscono le quattro operazioni CRUD.


```

import json
from pathlib import Path

DB = Path("devices.json")

def carica():
    """Legge l'intero database in memoria (dict)."""
    if not DB.exists():
        return {"_meta": {"versione": 1}, "dispositivi": []}
    with DB.open(encoding="utf-8") as f:
        return json.load(f)

def salva(dati):
    """Scrive l'intero database su disco."""
    with DB.open("w", encoding="utf-8") as f:
        json.dump(dati, f, indent=2, ensure_ascii=False)

def aggiungi(nome, ip, attivo=True):
    # CREATE
    dati = carica()
    nuovo_id = max([d["id"] for d in dati["dispositivi"]], default=0) + 1
    dati["dispositivi"].append(
        {"id": nuovo_id, "nome": nome, "ip": ip, "attivo": attivo})
    salva(dati)
    return nuovo_id

def trova(nome):
    # READ
    dati = carica()
    return [d for d in dati["dispositivi"] if d["nome"] == nome]

def aggiorna(id_disp, **campi):
    # UPDATE
    dati = carica()
    for d in dati["dispositivi"]:
        if d["id"] == id_disp:
            d.update(campi)
            break
    salva(dati)

def elimina(id_disp):
    # DELETE
    dati = carica()
    dati["dispositivi"] = [d for d in dati["dispositivi"]
                           if d["id"] != id_disp]
    salva(dati)

```

Uso del modulo nell'applicazione

```

aggiungi("Robot-Saldatura", "192.168.1.30") # nuovo record
print(trova("PLC-Linea1"))                  # -> [{...}]
aggiorna(2, attivo=True)                    # modifica un campo
elimina(1)                                   # rimuove per id

```

Scrittura atomica: non corrompere il file

La `salva()` qui sopra ha un difetto: se il processo si interrompe a metà scrittura, il file resta troncato e `json.load` fallirà al riavvio. La pratica consolidata è scrivere su un file temporaneo nella stessa cartella e poi **rinominarlo** con `os.replace`, operazione atomica sui filesystem moderni: a valle resta sempre o il vecchio file integro o il nuovo completo, mai una via di mezzo.^[7]

```
import json, os, tempfile
from pathlib import Path

def salva_atomico(dati, percorso):
    percorso = Path(percorso)
    # file temporaneo nella STESSA cartella (stesso filesystem)
    with tempfile.NamedTemporaryFile(
        "w", encoding="utf-8", dir=percorso.parent,
        prefix=percorso.name + ".", suffix=".tmp",
        delete=False) as tmp:
        json.dump(dati, tmp, indent=2, ensure_ascii=False)
        tmp.flush()
        os.fsync(tmp.fileno())          # forza la scrittura fisica su disco
        nome_tmp = tmp.name
    os.replace(nome_tmp, percorso)      # rinomina atomica: vecchio 0 nuovo
```

Sostituendo la `salva()` con `salva_atomico()` il "database" diventa resistente a crash e interruzioni. Per maggiore robustezza si aggiunge spesso una copia `.bak` e una lettura con fallback.^[8]

In alternativa: TinyDB (libreria pronta)

Se non si vuole scrivere il CRUD a mano, **TinyDB** è un database documentale scritto in puro Python che salva tutto in un file JSON, con API di tipo NoSQL e query componibili; nessun server, nessuna configurazione.^[2,3]

```
pip install tinydb
```

```
from tinydb import TinyDB, Query

db = TinyDB("devices.json")          # un file JSON come database
Disp = Query()

# CREATE
db.insert({"nome": "Robot-Saldatura", "ip": "192.168.1.30", "attivo": True})

# READ (query componibili)
db.search(Disp.attivo == True)
db.search((Disp.nome == "PLC-Linea1") & (Disp.attivo == True))

# UPDATE
db.update({"attivo": False}, Disp.nome == "Inverter-Pompa")

# DELETE
db.remove(Disp.ip == "192.168.1.10")
```

LIMITE CRUCIALE: CONCORRENZA

TinyDB e i "database su file JSON" **non sono sicuri per scritture concorrenti** da più processi o thread: due script che scrivono insieme producono conflitti, e TinyDB non offre garanzie ACID.^[3,4] Inoltre la sua cache delle query non rileva modifiche fatte al file da processi esterni (serve `db.clear_cache()`).^[2] Per app web o accessi concorrenti, partire direttamente da SQLite.

13 Sviluppi futuri

- **Schema-first.** Lo schema diventa il contratto da cui partire, come si progetta uno schema di database prima del codice: abilita test automatici, suite di regressione e confronti tra versioni di modello.
- **Output dell'IA tipizzati e testabili.** Con la decodifica vincolata, i dati restituiti dai modelli sono verificabili come qualsiasi API: l'integrazione LLM smette di essere "esegui e guarda a occhio".
- **Grammatiche sempre più efficienti.** La ricerca sui motori di vincolo (Earley parser, pruning dinamico, caching degli schemi) punta a costo per-token trascurabile anche con schemi complessi.
- **JSON Schema verso la stabilità.** La versione "v1/2026" consolida un nucleo stabile con garanzie di compatibilità, riducendo la frammentazione tra draft.
- **JSON resta dominante.** Nonostante alternative binarie più compatte (CBOR, MessagePack), la leggibilità e l'ubiquità mantengono JSON al centro dell'interscambio dati e dell'interfaccia uomo-macchina-IA.

IN UNA FRASE

JSON evolve poco come *sintassi* (per scelta), ma moltissimo come *ecosistema*: lo schema e la generazione vincolata lo stanno trasformando nel contratto affidabile tra software tradizionale e modelli di IA.

14 Riferimento rapido

La grammatica in pillole

Elemento	Forma
Oggetto	{ "k1": v1, "k2": v2 }
Array	[v1, v2, v3]
Coppia	"chiave": valore
Valori ammessi	string · number · object · array · true · false · null
Stringa	"..." con escape \ " \\ \n \t \uXXXX
Numero	-? intero (.frazione)? ([eE][+-]?cifre)?

Ricorda (do / don't)

SÌ	NO
"doppi apici"	'apici singoli'
[1, 2, 3]	[1, 2, 3,]
0.5 30 1e3	.5 030 +7
null	NaN · Infinity · undefined
nessun commento	// ... /* ... */
ID grandi come stringa	ID grandi come numero

Strumenti da riga di comando e codice

```
# Validare e formattare con Python
python -m json.tool config.json

# jq: estrarre un valore annidato
jq ".plc.ip" config.json          # -> "192.168.1.10"

# JavaScript: parse / serialize
const obj = JSON.parse(testo);
const txt = JSON.stringify(obj, null, 2);

# Python: parse / serialize
import json
dati = json.loads(testo)
testo = json.dumps(dati, indent=2, ensure_ascii=False)
```

PROMEMORIA

Per leggere un JSON "a occhio" usa l'indentazione (2 spazi); per trasmetterlo in produzione usa la forma *minificata* (senza spazi). Valida sempre l'input ricevuto prima di usarlo: un JSON sintatticamente valido non è automaticamente *corretto* nei contenuti — per quello serve uno schema.

15 Fonti e riferimenti

Riferimenti normativi del formato e fonti consultate (giugno 2026) per le sezioni su JSON come base di dati, Python e intelligenza artificiale. Le risorse online si aggiornano nel tempo.

1. JSON — standard del formato: **RFC 8259 / STD 90** (IETF) ed **ECMA-404**. rfc-editor.org/info/std90
2. TinyDB — documentazione ufficiale (storage JSON, query, cache). tinydb.readthedocs.io
3. Real Python — *TinyDB: A Lightweight JSON Database for Small Projects*. realpython.com/tinydb-python
4. M. Nealer — *TinyDB: A Fast and Easy Document DB without a Server (no ACID)*. medium.com/@marcnealer
5. StackShare — *JSONlite vs SQLite* (concorrenza in scrittura). stackshare.io/stackups/jsonlite-vs-sqlite
6. Educative — *Storing Data with SQLite* (limiti dei file JSON; SQLite incluso in Python, tipi e indici). educative.io
7. TheLinuxCode — *Python os.replace() for Safe, Atomic File Updates*. thelinuxcode.com
8. K. (DEV Community) — *Crash-safe JSON at scale: atomic writes + recovery without a DB*. dev.to/constantina
9. Python — documentazione del modulo `json`. docs.python.org/3/library/json.html
10. JSON Schema — specifica e dialetti (Draft 2020-12; v1/2026 in arrivo). json-schema.org/specification
11. OpenAI — *Introducing Structured Outputs in the API* (constrained decoding). openai.com
12. Let's Data Science — *How Structured Outputs and Constrained Decoding Work* (XGrammar, Ilguidance). letsdata-science.com
13. buildmvpfast — *JSON Mode vs Function Calling vs Structured Output: 2026 Guide*. buildmvpfast.com